

현대적 저수준 그래픽스 API 기반 데이터 지향 멀티플랫폼 렌더링 프레임워크 dy_framework

팀 명	DyD	학 과	컴퓨터소프트웨어공학과 C반
팀 원	한재승(팀장), 윤훈, 정현진, 정준혁		
지도교수	정석용		

1. 작품의 개요 - 개발 배경과 필요성

AI 시대의 핵심 연산 자원, GPU.

그 가능성을 활용하려면 개발 기반도 달라져야 한다.

인공지능의 학습과 추론, 과학 시뮬레이션, 실시간 그래픽스까지 GPU를 활용하는 분야가 넓어지고 있다. 대량의 데이터를 빠르게 계산하는 GPU의 성능을 끌어내려면, 소프트웨어도 데이터의 배치와 전달, 작업의 실행 순서를 함께 설계해야 한다.

그러나 GPU를 세밀하게 제어하는 Vulkan·Direct3D 12·Metal은 자원 관리와 명령 실행 방식이 서로 다르다. 여러 플랫폼을 지원하려면 각 API의 규칙을 이해하고 구현을 따로 유지해야 한다.

여러 운영체제에서 실행되는 것과 각 환경의 GPU를 일관된 방식으로 제어하는 것은 서로 다른 과제다. 플랫폼 간 이식성을 확보하면서도 GPU 자원과 실행에 대한 제어권을 유지할 수 있는 공통 기반이 필요하다.

dy_framework는 이러한 차이를 다루는 C++ 렌더링 프레임워크다. GPU 자원과 명령을 직접 제어할 수 있는 공통 RHI를 제공하고, Vulkan·Direct3D 12·Metal의 구현을 백엔드로 분리했다.

2. 작품의 구성

구성 요소	구현 내용
장면 데이터 관리	메시·재질 참조와 변환 정보를 용도별 배열로 관리하고 GPU 입력 데이터를 구성한다.
공통 GPU 제어	버퍼·텍스처·명령 목록·자원 상태·동기화를 공통 RHI로 제공한다.
플랫폼별 구현	Vulkan·Direct3D 12·Metal 각각의 자원 관리와 명령 실행 규칙을 구현한다.

3. 프로젝트 개발환경 및 개발자

개발도구 및 사용 언어

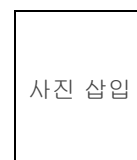
C++17, CMake
MSVC / GCC / Clang
HLSL, GLSL, Metal
Windows, Linux, macOS



한재승
Core · RHI



윤훈
Vulkan



정현진
Direct3D 12



정준혁
Metal

4. 핵심기술 설명

1) 현대적 저수준 API의 제어 모델을 공통 구조로

현대적 그래픽스 API에서는 GPU 자원의 상태와 수명, CPU와 GPU 사이의 동기화를 개발자가 직접 관리한다. dy_framework의 RHI는 버퍼·텍스처, 그래픽스·컴퓨트 명령, 제출과 완료 확인을 공통 인터페이스로 제공한다. 상위 시스템은 이를 바탕으로 필요한 GPU 작업과 실행 순서를 설계한다.

2) 메모리 접근을 고려한 데이터 배치

같은 계산도 데이터가 메모리에 놓인 방식에 따라 처리 효율이 달라진다. 위치만 읽는 연산이라면 필요한 위치값을 모아두어 한 번에 가져온 데이터를 더 많이 활용할 수 있다. 데이터 지향 설계(DOD)는 실제 연산과 데이터의 접근 방식에서 프로그램 구조를 정하는 접근이다.

CPU 캐시 예시 | 위치 P = 16B, 캐시 라인 = 64B

AoS · 객체별 배치

P1	기타 속성 48B
P2	기타 속성 48B
P3	기타 속성 48B
P4	기타 속성 48B

SoA · 속성별 배치

P1	P2	P3	P4
----	----	----	----

P1을 가져오면 P2·P3·P4도 함께 적재

기타 속성은 별도 배열에 보관

이번 위치 연산에서는 읽지 않음

접근 순서 P1 → P2 → P3 → P4

MISS	MISS	MISS	MISS
------	------	------	------

4라인·256B 적재 / 64B 사용

접근 순서 P1 → P2 → P3 → P4

MISS	HIT	HIT	HIT
------	-----	-----	-----

1라인·64B 적재 / 모두 사용

MISS는 새 라인 적재, HIT는 이미 적재된 라인에서 읽기. 위치 블록의 첫 접근을 순서대로 표시.

64B 정렬·초기 미적재, 프리페치·추출 제외. 개념 예시. 참고: Intel, Memory Layout Transformations.

3) GPU의 발전과 함께 달라진 소프트웨어 설계

GPU의 연산 능력이 커지면서, 데이터를 준비하고 전달하는 과정도 성능을 결정하는 중요한 문제가 되었다. 필요한 데이터를 찾느라 메모리를 반복해서 읽거나, 작은 작업을 지나치게 나누어 전달하면 GPU의 처리 능력을 충분히 활용하기 어렵다. 현대 렌더러가 데이터의 배치와 작업의 묶음을 함께 설계하는 이유다. 앞서 살펴본 데이터 지향 설계도 이러한 요구와 맞닿아 있다.

작게 나누어 전달

A1	→ 명령 A
B1	→ 명령 B
A2	→ 명령 A
B2	→ 명령 B

데이터 4개 · 명령 4회

같은 연산끼리 모아 전달

A1 A2	→ 명령 A
B1 B2	→ 명령 B

데이터 4개 · 명령 2회

A·B는 서로 다른 연산, 숫자는 입력 데이터. 함께 처리 가능한 독립 작업을 묶은 개념 예시.

설계 참고: Mike Acton, CppCon 2014 / NVIDIA, GPU Performance Background.

Vulkan·Direct3D 12·Metal은 개발자가 메모리와 명령, 동기화를 직접 관리할 수 있도록 제어 범위를 넓혔다. 그만큼 하드웨어에 맞춘 설계가 가능해졌지만, 플랫폼마다 다른 실행 규칙도 개발자가 다뤄야 한다.

dy_framework는 이 차이를 공통 RHI에 담았다. GPU 자원과 명령을 직접 다룰 수 있는 범위를 유지하면서, 플랫폼별 구현을 분리하는 것이 작품의 주요 설계 과제다.